Final Report of Group UG13

Project: Block Model Compression Algorithm

Student Name: Chih-Hao Chang

Student ID: a1818124

Team Members:

1. a1837374 Mengbo Ren
2. a1837306 Suchang Liu
3. a1824243 Yuyao Chen
4. a1837254 Yang Shi
5. a1832623 Hin Kai Chan
6. a1860970 Mohsan Salehi-Pugsley
7. a1818124 Chih-Hao Chang

Project Vision Reflection

Our initial project vision for the Maptek project was to create a single executable program that can receive geographical data in the specified plaintext stream format (with plaintext data, structured headers, and a tag table) and produce lossless compressed output. The most critical client requirement was compression ratio, which defines the program's compression efficiency. Other technical requirements, in descending order of priority, were speed performance, resource utilization (using parallel processing and GPU acceleration), robustness (through error handling, testing, and validation), maintainability, and scalability. The non-technical requirement was to provide comprehensive documentation. Scrum methodology was adopted to manage the project effectively.

The requirements have been clearly articulated in the client's documentation from the beginning of the project, and the client has not made substantial changes to them during the development of the project. This facilitated a shared vision between the client and the development team and a consistent focus on the ultimate objective. For instance, the input and output specifications as well as the main goal of improving the compression ratio and speed (evaluated through Titan testing machine) remained unchanged throughout the development phase.

Nevertheless, certain adjustments to the project vision were necessary during project development due to resource limitations and unforeseen challenges. For example, to improve the software's robustness, input validation was initially planned to be implemented by the development team to detect and handle input errors. However, it was re-prioritized from a mandatory requirement to an additional feature to be assessed after the core functionalities were completed, due to the following reasons: Firstly, the team found two limitations of input validation during scrum meetings: limited availability of real geographical plaintext inputs make the implementation and testing of validator challenging; and the validation overhead can compromise the speed performance of the program. Secondly, the client suggested that inputs from Titan testing machine may be assumed correct in a later sprint retrospective.

In summary, the project's vision has remained largely unchanged due to client's comprehensive initial documentation. However, minor adjustments were made to address development challenges while preserving the project's main objectives.

Customer Q&A Reflection

Here are some of the most important questions asked and the corresponding answers (including explanation of their significance to the final project):

Q1: Is loss a criterion like processing speed and compression ratio, which is to be improved (reduced) as we develop the project?

A1: No, loss must be zero (lossless). This is a crucial software requirement to be fulfilled before attempting to improve the processing speed and the compression ratio and is not a criterion to be improved on overtime. In other words, regardless of the speed and compression performance, if there is loss in the compressed output, we deemed that the deliverable does not meet the requirement.

Q2: Is the block compression expected to work with 2D model, or 3D model only, since the Z-count is 1 in the example provided in the product description?

A2: The product is expected to compress 3D model, and thus only 3D input would be tested by the Titan machine. However, the development team may consider it beneficial to start with implementing a compression algorithm using lower dimensional parent block, and then generalize it to accommodate higher dimensional parent blocks, to capture higher-order dependencies in the data. This follows the agile approach of Scrum methodology, as we aim to improve each deliverable throughout the develop phase, rather than implementing the entire project all at once (from nothing to done) like the waterfall methodology.

Q3: Are we expected to create our own input stream for testing?

A3: No, Titan testing machine provides test files for various competition (namely The Intro One, The Fast One, and The Worldly One). They are quite complicated, so it is recommended to download them for local testing. This ensures that they would execute as expected on Titan testing machine. However, it can be helpful to develop simple local testing during implementation (agile approach).

Q4: What is the scrum master’s job and how is it assigned?

A4: The development team assigns the role of scrum master to the team members in a balanced and rotational manner across the members, ensuring equal distribution across the number of sprints. The scrum master’s responsibility is mainly to facilitate collaboration between team members, such as evenly allocating the workload to members, minimizing idle time, and resolving deadlocks between members. This contributes to the quality of the final project, because the rotational assignment of the scrum master role allows all team members to showcase their diverse leadership skills and unique approaches to facilitate the team. Moreover, each member’s scrum master’s experience provides valuable insights for subsequent scrum masters to adapt successful practices and to further improve collaboration. Finally, this helps

prevent burnout, as no single member is burdened with the role for the entire development phase.

Q5: What happens if a ticket is not done in a sprint (spillover)?

A5: The tickets that are in the sprint backlog, the to-do section, and the in-progress section at the end of each sprint should be moved back to product backlog. This is important for the software development process, because the business value, priority, and resources allocated can be reevaluated in the context of the project's current goals. Moreover, returning spillover ticket to the product backlog can help prevent the accumulation of technical debt and identify the potential bottlenecks to be discussed by the team.

Here is the reflection on how the client meetings from my personal perspective:

Our meetings with the client ran smoothly and were highly valuable to the software's development. At the start of each meeting, the Scrum master would outline our previous sprint's goals, the strategies used to achieve them, and our current progress. Then the client would provide feedback based on that and the project board. Lastly, the remaining time would be used to discuss specific challenges which the client's suggestions could be helpful for.

One aspect we could have done differently was constructing a more efficient project board early in the development phase. After our second client meeting, we realized that without an efficient project board, it's hard to inform our progress in client meetings with time constraints. An improved project board would allow the client to quickly grasp the project status and make relevant suggestions. For example, linking tasks (tickets) to user stories would help the client understand how each task contributes to the broader objectives. Furthermore, tasks should be more well-defined. This includes a description of each assigned member's role to justify resource allocation, a definition of done that is clear and assessable, and optionally an acceptance, which provides a more technical definition of done. Thus, agreeing with the client early on about the project board format could have streamlined future client meetings, allowing more time for in-depth discussions on the technical aspect of complicated features.

These client meetings have taught me plenty things that can be applied for future projects. Firstly, despite the project description provides well-defined requirements, client meetings remain essential because not every requirement and challenge can be anticipated. When uncertainties arise, it's crucial to consult the client for alignment, as they are the best position to guide the expectations. Secondly, these meetings provide insights into the client's priorities, helping the team adjust its strategy to align with the client's vision for the project. Thirdly, client feedback also offers valuable domain-specific knowledge, allowing the team to make well-informed decisions and assumptions about the project without wasting much time. Last but not

least, keeping in contact with the client ensure the project stays on track, which helps high quality deliverables and prevent the risk of having big changes that are time-consuming.

Users and User Stories (Group)

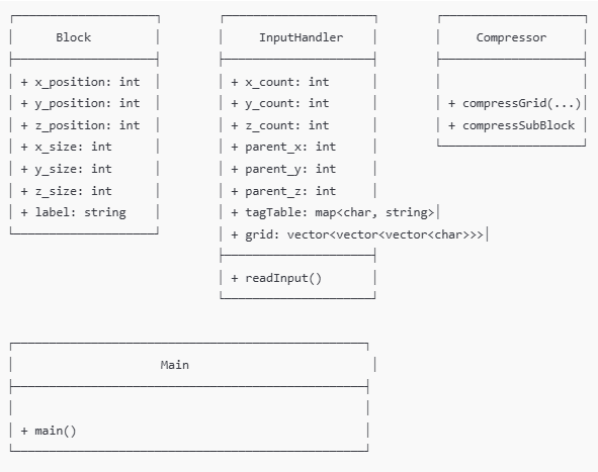
Developer: Responsible for implementing and optimizing algorithms.
Key actions include debugging tools like 2024_runnerA.py, writing shell scripts for deployment testing, writing the RLE algorithm and refining algorithms like the 333 compression.
Important user stories:
(1) improving compression through a 26-way tree for the 333 algorithm, and
(2) achieving multithreaded processing to boost efficiency
(3) achieving RLE algorithm to achieve efficient data compression.

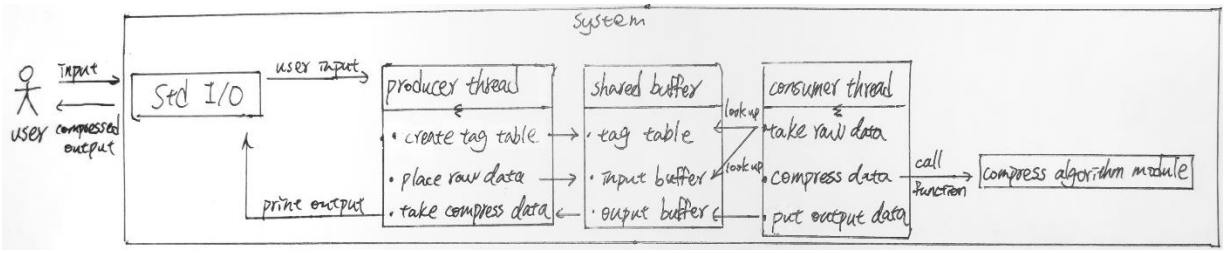
Tester: Primarily tasked with validating and benchmarking algorithms.
This role involves actions like testing on the Maptek website to gain rankings and using metrics for compression efficiency.
Key stories:
(1) obtaining performance benchmarks for the Worldly One dataset on Maptek,
(2) tracking improvements post-algorithm modifications, and
(3) using tools for testing and debugging code performance.

Project Manager: Oversees project progress, prioritizes tasks, and ensures deployment standards.
Responsibilities include managing sprint goals, documenting processes, and aligning team efforts.
Key stories:
(1) outlining code debugging processes for clarity,
(2) setting multithreading as a priority for efficient execution, and
(3) directing benchmark goals on the Maptek site.

In the initial report we identified two types of users, the developers and clients. However, as the project progressed, we realized our user identified was incorrect. Clients were unnecessary participants, while the roles of testers and project managers were missing but critical. We have found that testers play a vital role in verifying the efficiency of the algorithms, improving compression accuracy and performing benchmarking, which ensures the quality and reliability of our work. The project manager, on the other hand, is critical to monitoring progress, setting priorities and maintaining quality standards, and ensuring that project objectives are achieved on time. Thus, our user roles evolved from the initial developer-client structure to a more organized team setup of developers, testers, and project managers.

Software Architecture





Modular Monolithic Architecture is the architecture selected for the project after analysing the client's requirements. This approach combines Monolithic Architecture, where the software remains a single unit with one codebase, and Modular Architecture, where components are divided into independent modules. This modular approach reduces dependencies between components, improving maintainability, extensibility, and testability.

Using a traditional Monolithic Architecture ensures a single codebase, which aligns with the client’s requirement for a unified software unit. However, monolithic systems often face challenges with maintainability and extensibility due to tightly coupled components. To address this, we adopted a Modular Monolithic Architecture, where components are structured as loosely coupled modules. Each module handles a distinct set of functionalities, with defined boundaries and interfaces. This enhances separation and independence.

The loosely coupled modules in this architecture make it easier to test individual components, as each module can be tested in isolation. This separation of responsibilities also improves code readability, as developers can work within specific modules without needing to understand the entire system. Additionally, because modules are designed to be self-contained, changes within one module minimally impact others, which makes the system more extensible than a traditional monolithic setup.

Given the client's requirements for threading acceleration, this feature have been incorporated into our architecture. The client's specification of a two-thread setup led us to design a producer-consumer module. In this setup, the producer thread handles I/O and feeds data into a shared buffer, while the other consumer thread processes the data by performing compression. This structure improves CPU utilization by reducing the coordination overhead between threads and focusing each thread on a specialized task, thus enhancing performance. If both threads were involved in compression and I/O simultaneously, CPU efficiency would likely drop due to increased coordination overhead.

In practice, the Producer thread captures input data and generates a tag table, placing data requiring compression into an input data buffer. The consumer thread then accesses this buffer, applying the compression by calling functions from the compression algorithm module.

Tech Stack and Standards

Final tech stack for our application (back-end) (there is no front-end):

- Programming language: C++
- Standard Template Library (STL): string, unordered_map, vector
- Build tools: Makefile
- Version Control: Git, GitHub Enterprise
- Unit Testing: manual Bash script

Tools for communication and development:

- Project Management: GitHub Projects.
- Documentation: GitHub Wiki
- Communication: Microsoft Team, Discord, Wechat
- IDE: VS Code
- Linter: clang-format

Coding Standards:

- Google C++ Style Guide, following the Uncle Bob clean code guideline (http://cleancoder.com/products)

Justifications and Reflection:

The chosen tech stack for the back-end of our application reflects a strong alignment with the project's high-performance requirements, particularly in handling large datasets and implementing complex compression algorithms. C++ was selected as the programming language for its efficiency in system-level programming, allowing fine control over memory and computational resources. This choice proved beneficial for optimizing data compression tasks, offering a level of performance that higher-

level languages like Python could not achieve. Although initially proposed in the report, C++ proved especially effective when handling computationally heavy workloads, supporting the project's processing requirements seamlessly.

Version control was managed through Git and GitHub Enterprise, both of which were initially proposed in the report. GitHub Enterprise was instrumental in providing secure and efficient collaboration, allowing team members to work on different parts of the codebase simultaneously without conflict. Its built-in security features and seamless integration with other tools proved valuable for maintaining a clean and organized codebase, particularly when coordinating with the client. One area for potential improvement, however, was GitHub's limited support for some larger data files, which occasionally required workarounds but did not significantly impact overall project progress.

For unit testing, we opted to use manual Bash scripts as outlined in the initial plan. While automated frameworks could provide more advanced testing capabilities, Bash scripts allowed us to tailor tests to our unique concurrency needs, ensuring flexible, repeatable tests across a Linux environment. This approach worked well for our C++ codebase, where simplicity and direct access to the terminal were key. However, the lack of detailed logging that some automated testing tools provide meant that troubleshooting occasionally took longer than expected. Nonetheless, the simplicity of manual scripts fit our needs without introducing additional complexity.

In terms of project management and documentation, we utilized GitHub Projects and GitHub Wiki, both of which were proposed in the initial report. GitHub Projects proved effective for task tracking and transparency, allowing the client to stay updated on development status, while GitHub Wiki kept all technical documentation centralized and accessible. These tools supported a seamless flow from code to documentation and task management, ensuring all materials were easily accessible. The only minor drawback we encountered was GitHub's lack of advanced project tracking features, such as more detailed timelines or automated workflows, but this did not significantly hinder our progress.

For communication, we relied on Discord, WeChat, and Microsoft Teams, which were specified in the initial report. Discord and WeChat facilitated fast, informal discussions among team members, while Microsoft Teams provided a structured setting for more formal conversations with the client. Each tool served its purpose effectively, though occasionally switching between platforms led to missed messages and minor delays in response time. Nevertheless, the diverse communication channels helped address the varying preferences of team members and clients.

Our development environment, Visual Studio Code, remained consistent with the initial plan and proved to be a versatile and reliable choice. Its extensions and customization options enabled smooth integration with Git and debugging tools, streamlining our workflow. We also implemented clang-format as our linter, following the Google C++ Style Guide and Uncle Bob's Clean Code principles, which were

recommended in the initial report. These coding standards provided a solid foundation for maintaining clean, readable, and maintainable code, while clang-format ensured consistent formatting across the team.

Overall, the final tech stack required no major deviations from the initial report, which indicated the foresight and suitability of the original proposal. Most choices worked as expected, though minor areas like project tracking and unit testing logging could be improved in future projects. This tech stack successfully supported our goal of delivering a high-quality application that met the client's needs, maintaining both performance and manageability throughout development.

Group Meetings and Team Member Roles

The frequently and duration we (virtually) meet:

We initially decided to meet in-person every Tuesday at 10:00 a.m. at university's Hub Centre for two hours and mainly focused on code-related group discussion. Face-to-face meetings was preferred as it generally offers better communication and collaboration. However, as time moves on, we found it challenging to coordinate team members with varying schedules and the out-of-town member. The team members became progressively busier as the semester progressed as well. Therefore, we transitioned to online meetings conducted on Discord and Wechat to accommodate everyone. This provides not only flexibility of location, but also flexibility of time, as team members are no longer obligated to travel to the university. Thus, we could increase the frequency of the meeting from once a week to two or three times a week (the time is scheduled each week according to team members' schedule to provide flexibility) and reduce the duration to one hour (due to increased frequency). There were disadvantages for this change as well, such as less direct engagement and minor technical issues, such as internet delays and team members being unfamiliar with how to operate the communication. However, the overall benefits outweighed the drawbacks, as we could ensure more team member would be available for the shorter and more frequent online meetings, and deadlocks could be discussed and resolved more promptly, reducing the team member's idle time they cause.

The time scheduled for the sprint retrospective meetings:

The client arranged our retrospective meetings once every two weeks (start of each sprint) on Wednesday at 5:30 p.m. Each meeting lasted 30 minutes and was conducted online via Zoom. This schedule matches the availability of our team to the timing preferences of our proxy client. Retrospective meetings facilitate open communication between developers and clients, thereby providing us with valuable insights, as we received direct client feedback and additional requirements that were uncovered. Such feedback was essential in refining our product to meet client

expectations more accurately, and thus, it play an important role in the Agile framework.

The arrangement for additional feedback channels with the customer:

Microsoft Teams were used to communicate with the client to provide feedback outside of the sprint retrospective meetings. We prioritized quality over quantity in our communication approach. Thus, we did not establish additional communication channels with our customers, as we deemed the retrospective meetings and Microsoft Teams group chat are efficient for feedback. Additionally, adding more channels could lead to miscommunications or redundancies, because multiple communication channels for the same purpose makes it harder to manage for both our team and the client, and duplicate or unanswered questions can cause confusion.

The Scrum Masters for each sprint:

- 1st sprint: a1860970 Mohsan Salehi-Pugsley
- 2nd sprint: a1837306 Suchang Liu
- 3rd sprint: a1837374 Mengbo Ren
- 4th sprint: a1818124 Chih-Hao Chang
- 5th sprint: a1824243 Yuyao Chen, a1837254 Yang Shi, a1832623 Hin Kai Chan

Personal reflection on group meetings and own role:

Our team meetings served as dedicated spaces for technical discussions, sprint planning, and sprint reviews.

Typically, we used these sessions to discuss the technical aspect of the project. Members responsible one module might not have full visibility into others since we implemented a modular architecture, and this could lead to system incompatibilities. Team discussions helped address this by allowing members to share insights about how their modules impacted the rest of the system and to gained context on potential challenges, which enables a more in-depth understanding and consistent solutions across the system.

Discussion on the project’s current progress and reviewing the sprint backlog during meetings are crucial as well. If a specific concern emerged, we would note it for discussion in the upcoming sprint retrospective meeting. I believe that it would be beneficial however, to outline the meeting objectives and the scope of discussion before each meeting, so that we could have a clear goal and avoid the topic diverging from the objectives, which could cause conversations exceeding the allocated time.

In my role, I focused on researching and implementing technical solutions as well as managing the project board. During meetings, I typically guide discussions on the technical implementation of the compression algorithm and the compatibility across modules to ensure high-quality implementations across the system.

Appendix: Snapshots

Snapshot Week 5 of Group Block-UG13

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

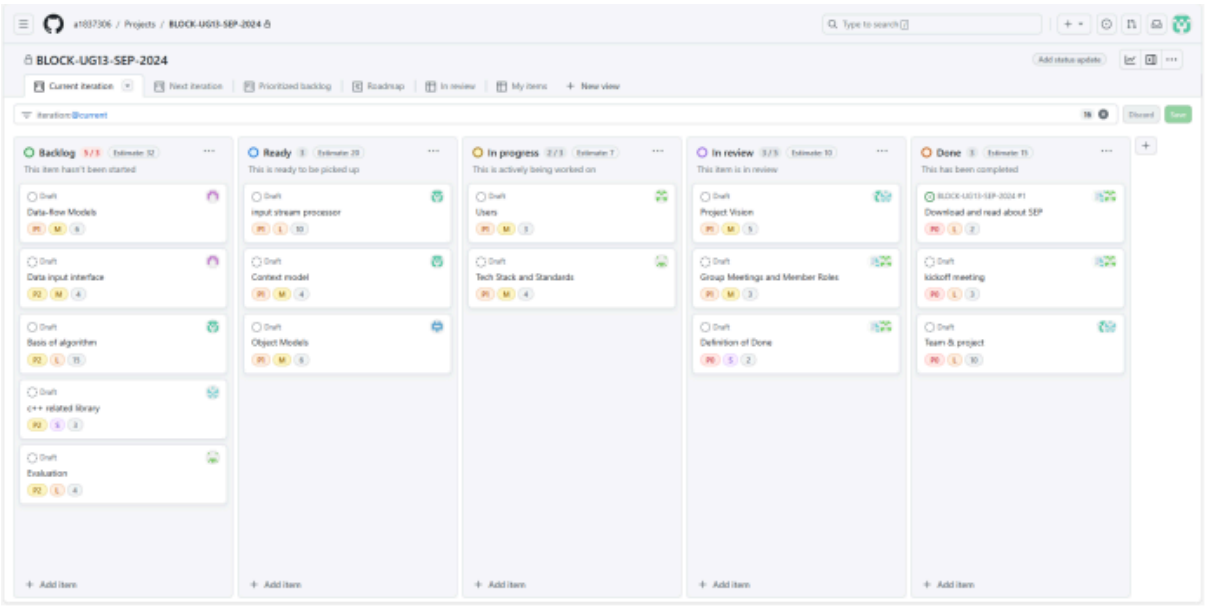
a1837254 Yang Shi

a1832623 Hin Kai Chan

a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

Product Backlog and Task Board:



Sprint Backlog and User Stories:

Sprint Overview

For the current sprint, we have selected the following user stories to focus on, catering to both developers and clients:

1. As a developer, I want to have a clear and concise API documentation so that I can quickly understand and implement the block model compression algorithm.

- Task 1.1: Create detailed API documentation for the compression algorithm.

- Task 1.2: Ensure documentation covers all public methods and parameters.

- Task 1.3: Provide code examples for common use cases.

2. As a client, I want to be able to easily import my block models into the program so that I can begin the compression process without any hassle.

- Task 2.1: Design a user-friendly import feature for block models.

- Task 2.2: Ensure compatibility with various block model formats.

- Task 2.3: Implement error handling for import issues.

These user stories and their related tasks will guide our development efforts for the current sprint, ensuring that we address the needs of both developers and clients in creating an effective block model compression program.

ID	Title	Description	Priority	Estimated Effort	Responsible
SB-001	Scrum tool	Register, try, compare and choose Scrum management tools	2	2	
SB-002	API documentation	Create detailed API documentation for the compression algorithm.	1	3	
SB-003	Methods & parameters	Ensure documentation covers all public methods and parameters.	1	4	
SB-004	Code examples	Provide code examples for common use cases.	2	5	
SB-005	Import feature	Design a user-friendly import feature for block models.	1	3	
SB-006	Compatibility	Ensure compatibility with various block model formats.	2	4	
SB-007	Error handling	Implement error handling for import issues.	2	5	

Definition of Done

1 Code complete:

All planned functionality has been implemented, and the code conforms to the team's coding specifications.

2 Code Review passed:

The code has been peer reviewed at least once and all issues raised have been addressed.

3 Compile pass:

Compile through, there are no errors reported, and there are no warnings that affect the operation.

4 Tests passed:

All relevant automated tests (e.g., unit tests, integration tests) pass, and test coverage meets the standards agreed upon by the team.

All relevant manual tests (e.g., functional tests, acceptance tests) passed and no new defects were found.

5 Performance tests passed:

If applicable, all relevant performance tests have been passed and the performance metrics meet the standards agreed upon by the team.

6 Documentation updates:

All relevant technical documentation (such as design documentation, API documentation) has been updated and is consistent with the code.

7 Code merged into the main branch:

The code has been merged into the main branch and no new conflicts or problems have been introduced.

ID	Title	Description	Priority	Estimated Effort	Responsible
PB-001	Download and read about SEP	Download and study the Scrum Event Planning guide	1	2	
PB-	kickoff meeting	Organize and conduct the project kick-off meeting	1	3	
PB-	Team & project	Assemble the Scrum team and select a suitable project	1	4	
PB-	Project Vision	Define the overall goal and desired outcome of the project	2	5	
PB-	Users	Identify and document the target users for the project	2	3	
PB-006	Context model	Create a context model to understand the environment and constraints of the project	3	4	
PB-007	Data-flow Models	Develop data flow models to visualize the flow of data within the system	3	5	
PB-008	Object Models	Design object models to represent the structure and behavior of the system	3	6	
PB-009	Tech Stack and Standards	Choose the technology stack and establish coding standards	2	4	
PB-010	Group Meetings and Member Roles	Schedule regular meetings and define roles and responsibilities for team members	2	3	
PB-	Data input interface	Plan the design and implementation of the data input interfaces	2	4	
PB-	Basis of algorithm	Establish the basis for the algorithms used in the project	4	5	
PB-	c++ related library	Research and select appropriate libraries for C++ development	3	3	
PB-014	Evaluation	Evaluate the progress and results of the project against the defined goals	5	4	

ID	Title	Description	Priority	Effort	Status			Responsible
PB-001	Download and read about SEP	Download and study the Scrum Event Planning guide	1	2			Done	
PB-002	kickoff meeting	Organize and conduct the project kick-off meeting	1	3			Done	
PB-003	Team & project	Assemble the Scrum team and select a suitable project	1	4			Done	
PB-004	Project Vision	Define the overall goal and desired outcome of the project	2	5			Done	
PB-005	Users	Identify and document the target users for the project	2	3		In Progress		
PB-006	Context model	Create a context model to understand the environment and constraints of the project	3	4		In Progress		
PB-007	Data-flow Models	Develop data flow models to visualize the flow of data within the system	3	5		In Progress		
PB-008	Object Models	Design object models to represent the structure and behavior of the system	3	6		In Progress		
PB-009	Tech Stack and Standards	Choose the technology stack and establish coding standards	2	4		In Progress		
PB-010	Group Meetings and Member Roles	Schedule regular meetings and define roles and responsibilities for team members	2	3			Done	
PB-011	Data input interface	Plan the design and implementation of the data input interfaces	2	4	To Do			
PB-012	Basis of algorithm	Establish the basis for the algorithms used in the project	4	5	To Do			
PB-013	c++ related library	Research and select appropriate libraries for C++ development	3	3	To Do			
PB-014	Evaluation	Evaluate the progress and results of the project against the defined goals	5	4	To Do			

Summary of Changes:

This is the first snapshot of the project and nothing has changed.

Snapshot Week 06 of Group Block-UG13

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

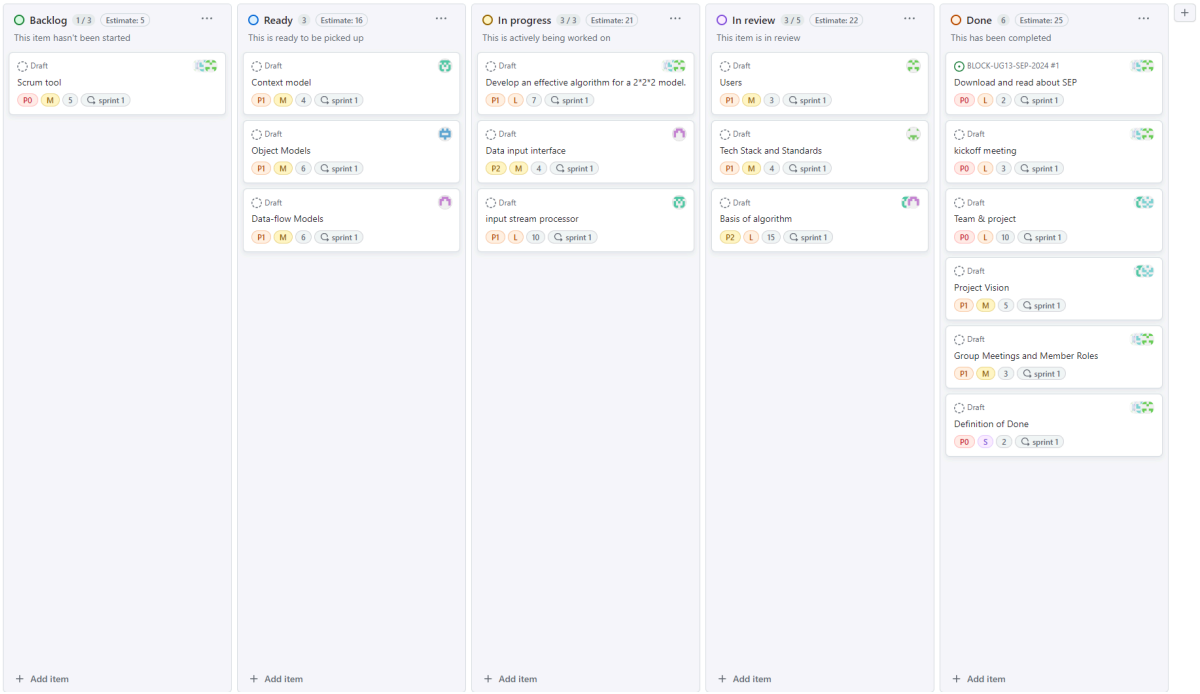
a1837254 Yang Shi

a1832623 Hin Kai Chan

a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

Product Backlog and Task Board:



Sprint Backlog and User Stories:

Sprint Overview

For the current sprint, we have selected the following user stories to focus on, catering to both developers and clients:

1. As a developer, I want to have a clear and concise API documentation so that I can quickly understand and implement the block model compression algorithm.

- Task 1.1: Create detailed API documentation for the compression algorithm.

- Task 1.2: Ensure documentation covers all public methods and parameters.

- Task 1.3: Provide code examples for common use cases.

2. As a client, I want to be able to easily import my block models into the program so that I can begin the compression process without any hassle.

- Task 2.1: Design a user-friendly import feature for block models.

- Task 2.2: Ensure compatibility with various block model formats.

- Task 2.3: Implement error handling for import issues.

These user stories and their related tasks will guide our development efforts for the current sprint, ensuring that we address the needs of both developers and clients in creating an effective block model compression program.

ID	Title	Description	Priority	Estimated Effort	Responsible
SB-001	Scrum tool	Register, try, compare and choose Scrum management tools	2	2	
SB-002	API documentation	Create detailed API documentation for the compression algorithm.	1	3	
SB-003	Methods & parameters	Ensure documentation covers all public methods and parameters.	1	4	
SB-004	Code examples	Provide code examples for common use cases.	2	5	
SB-005	Import feature	Design a user-friendly import feature for block models.	1	3	
SB-006	Compatibility	Ensure compatibility with various block model formats.	2	4	
SB-007	Error handling	Implement error handling for import issues.	2	5	

Definition of Done

Users can input an array of 3D block data into the program and expect the output to compress the input data. Currently, the expected input parent block size is 2*2*2.

1 Code complete:

All planned functionality has been implemented, and the code conforms to the team's coding specifications.

2 Code Review passed:

The code has been peer reviewed at least once and all issues raised have been addressed.

3 Compile pass:

Compile through, there are no errors reported, and there are no warnings that affect the operation.

4 Tests passed:

All relevant automated tests (e.g., unit tests, integration tests) pass, and test coverage meets the standards agreed upon by the team.

All relevant manual tests (e.g., functional tests, acceptance tests) passed and no new defects were found.

5 Performance tests passed:

If applicable, all relevant performance tests have been passed and the performance metrics meet the standards agreed upon by the team.

6 Documentation updates:

All relevant technical documentation (such as design documentation, API documentation) has been updated and is consistent with the code.

7 Code merged into the main branch:

The code has been merged into the main branch and no new conflicts or problems have been introduced.

Summary of Changes:

This week's snapshot mainly focused on tracking our latest progress on the backlog and task board. For example, we have identified some viable algorithms that are currently in the "review" phase, where everyone discussed the pros and cons of these algorithms. We also decided on the schedule for all meetings and assigned responsibilities to each team member. Additionally, we discussed our goals and began implementing the input stream processor and input interface, which are now marked as "in progress."

Snapshot Week 7 of Group **Block-UG13**

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

a1860970 Mohsan Salehi-Pugsley

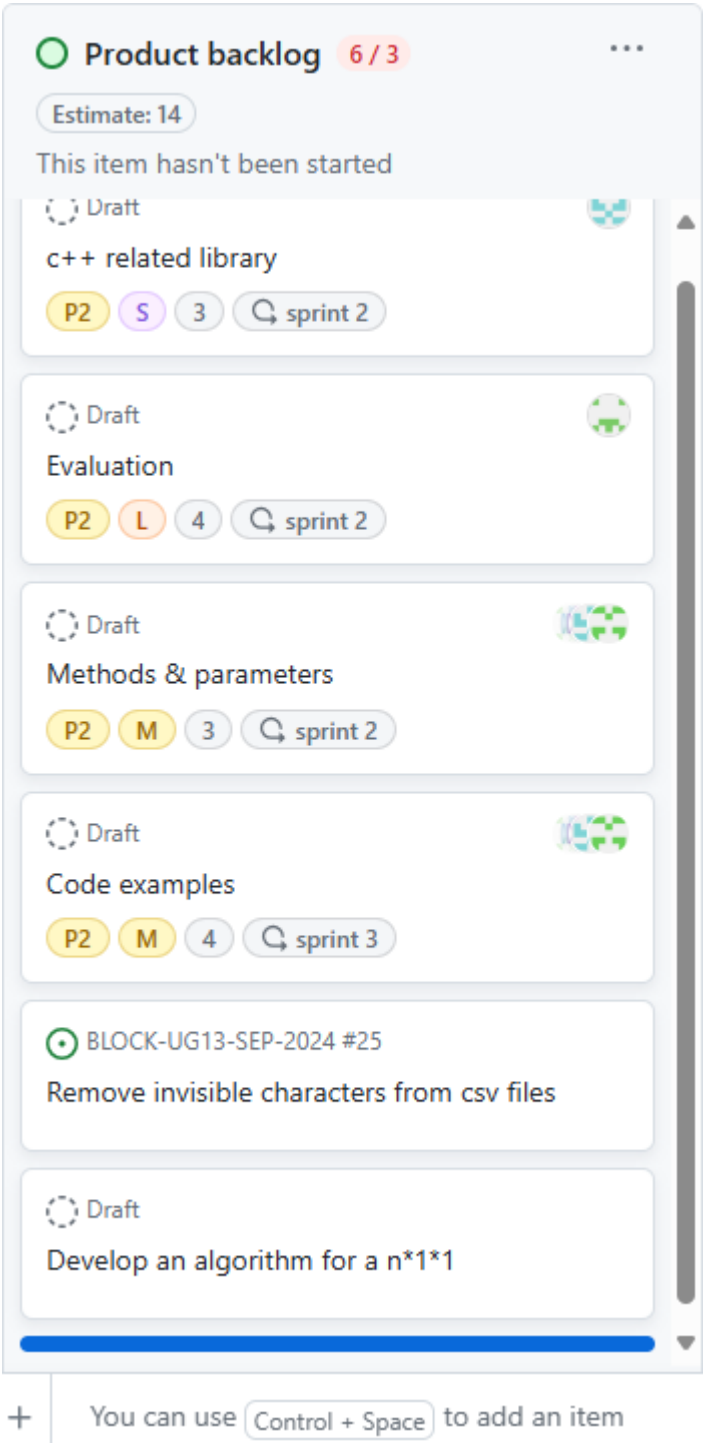
a1818124 Chih-Hao Chang

a1881772 Nauman Yawar Butt

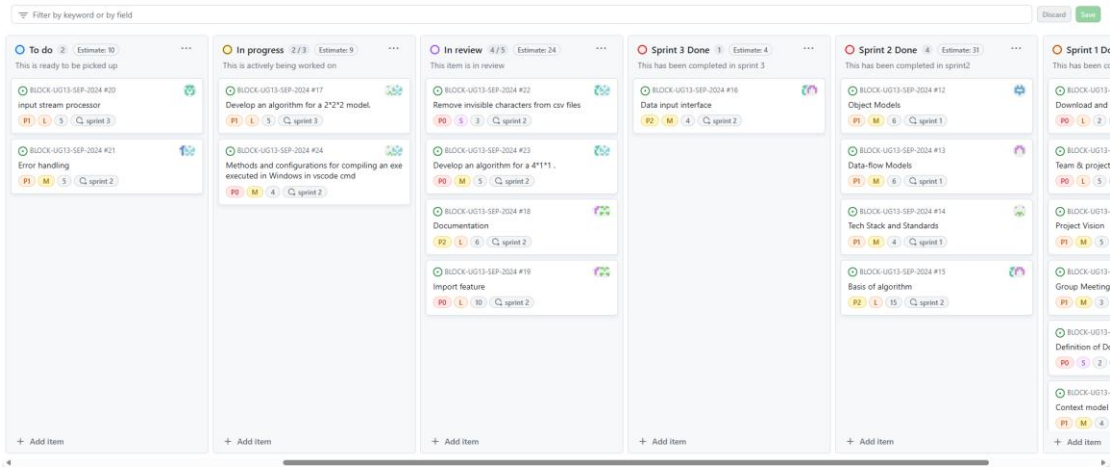
•

Product Backlog and Task Board:

The product backlog

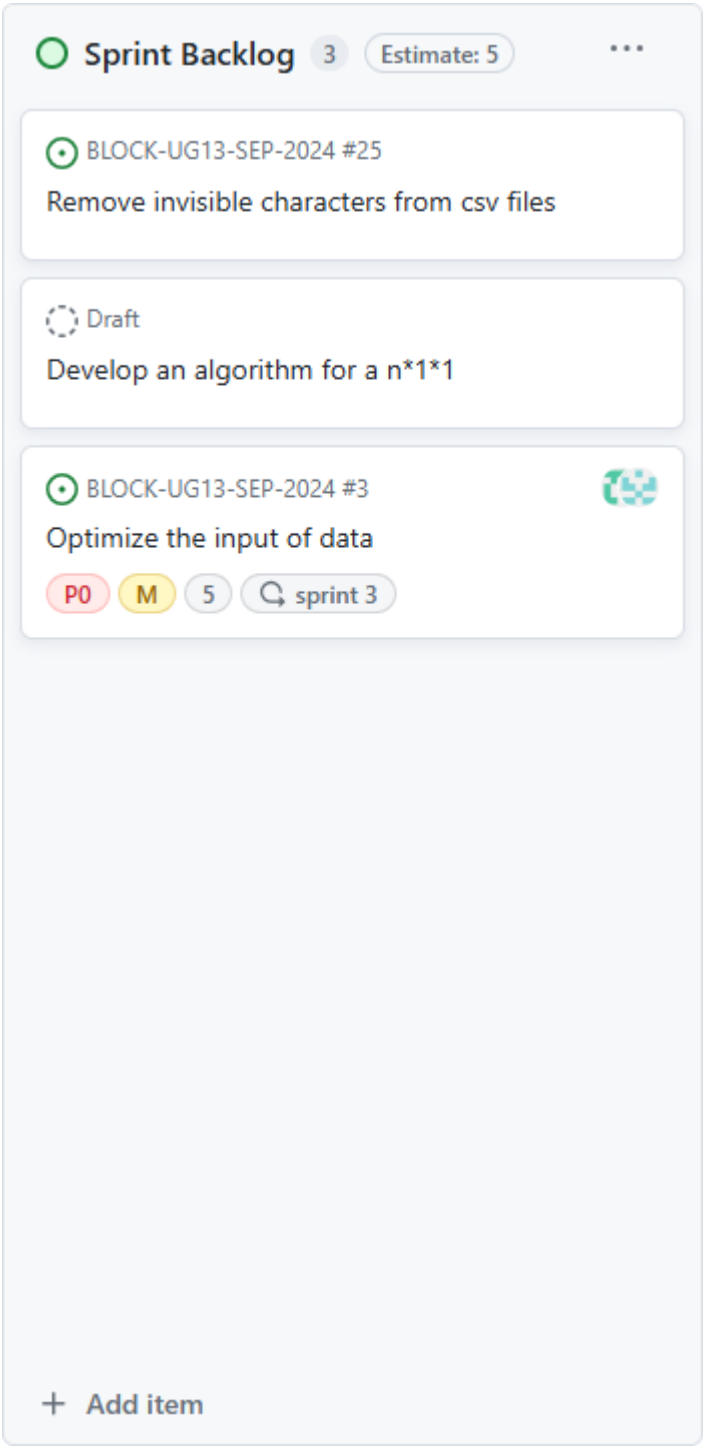


The task board



Sprint Backlog and User Stories:

Sprint backlog



user stories

Sprint Overview

For the current sprint, we have selected the following user stories to focus on：

1. As a developer, looking up line.empty() during debugging results in an error. Although the csv file looks like a blank line, it always doesn't work when determining conditions, resulting in errors.

2. As a developer, in order to familiarize yourself with the testing process, first implement a simple full version with compression capabilities in one direction.

3. Optimize data entry based on the results of previous sprints.A. File data.CVS input and stream; B. Check whether the printed data is consistent with the input data. C. By increasing the amount of data in the csv document, the test can run properly in stream mode.

These user stories and their related tasks will guide our development efforts for the current sprint, ensuring that we address the needs of both developers and clients in creating an effective block model compression program.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases

2 Tests passed:

The code passed the local test, conforming to logic and the output met expectations.

Summary of Changes:

1. Modified the form of the runtime data file name as a parameter;

2. Checked and deleted the characters that could not be printed and displayed in the data file, and corrected the process error;

3, change the compilation environment cmd, and install MinGW compiler, so that exe can run normally under windows

Snapshot Week 8 of Group Block-UG13

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

a1881772 Nauman Yawar Butt

Product Backlog and Task Board:

The product backlog

Product backlog 4 / 3

Estimate: 14

This item hasn't been started

Draft

New features

P2 S 3 sprint 3

Draft

Enhancement

P2 L 4 sprint 3

Draft

Technical Debt

P2 M 3 sprint 3

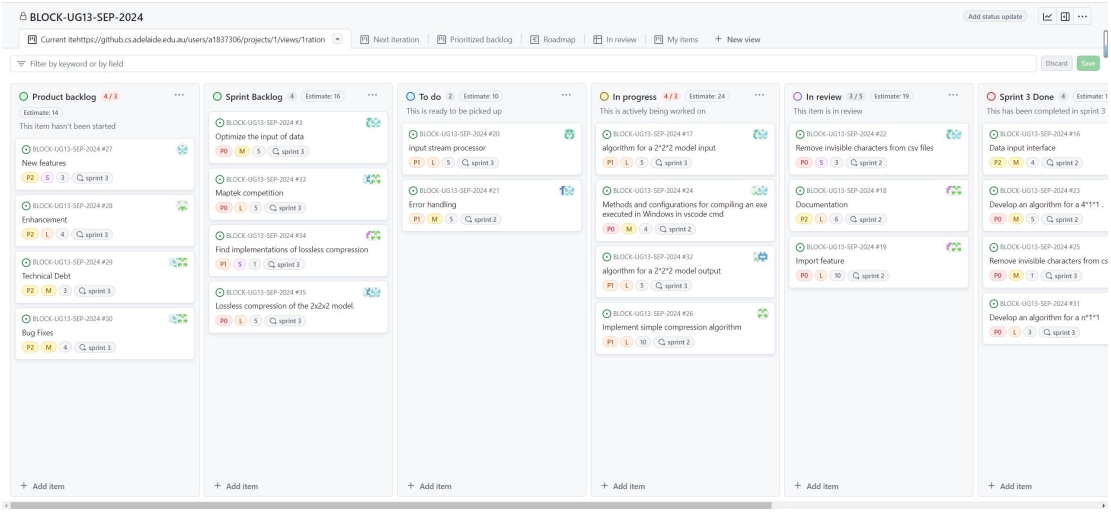
Draft

Bug Fixes

P2 M 4 sprint 3

+ Add item

The task board



Sprint Backlog and User Stories:

Sprint backlog

Sprint Backlog 4 Estimate: 16 ...

BLOCK-UG13-SEP-2024 #3

Optimize the input of data

P0 M 5 sprint 3

BLOCK-UG13-SEP-2024 #33

Maptek competition

P0 L 5 sprint 3

BLOCK-UG13-SEP-2024 #34

Find implementations of lossless compression

P1 S 1 sprint 3

BLOCK-UG13-SEP-2024 #35 ...

Lossless compression of the 2x2x2 model.

P0 L 5 sprint 3

+ Add item

User Stories

Sprint Overview

For the current sprint, we have selected the following user stories to focus on：

1. As a developer, looking up line.empty() during debugging results in an error. Although the csv file looks like a blank line, it always doesn't work when determining conditions, resulting in errors.
2. As a developer, in order to familiarize yourself with the testing process, first implement a simple full version with compression capabilities in one direction.
3. Optimize data entry based on the results of previous sprints.A. File data.CVS input and stream; B. Check whether the printed data is consistent with the input data. C. By increasing the amount of data in the csv document, the test can run properly in stream mode.

These user stories and their related tasks will guide our development efforts for the current sprint, ensuring that we address the needs of both developers and clients in creating an effective block model compression program.

Definition of Done

1 Code complete:
All planned functionality has been implemented, and the code conforms to the team's coding specifications.

2 Code Review passed:
The code has been peer reviewed at least once and all issues raised have been addressed.

3 Compile pass:
Compile through, there are no errors reported, and there are no warnings that affect the operation.

4 Tests passed:
All relevant automated tests (e.g., unit tests, integration tests) pass, and test coverage meets the standards agreed upon by the team.
All relevant manual tests (e.g., functional tests, acceptance tests) passed and no new defects were found.

5 Performance tests passed:
If applicable, all relevant performance tests have been passed and the performance metrics meet the standards agreed upon by the team.

6 Documentation updates:
All relevant technical documentation (such as design documentation, API documentation) has been updated and is consistent with the code.

7 Code merged into the main branch:
The code has been merged into the main branch and no new conflicts or problems have been introduced.

Summary of Changes:

1. Remove invisible characters from csv files
2. The RLE compression method was used for the 4*1*1 model, and the data was traversed. When encountering the same value, the count was recorded, and when the value changed, the current value and the count pair were stored in the result.
3. Try using the Quantization of Vertex Positions method to compress 3D models without loss of quality. This algorithm is still being refined.

Snapshot Week 9 of Group **Block-UG13**

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

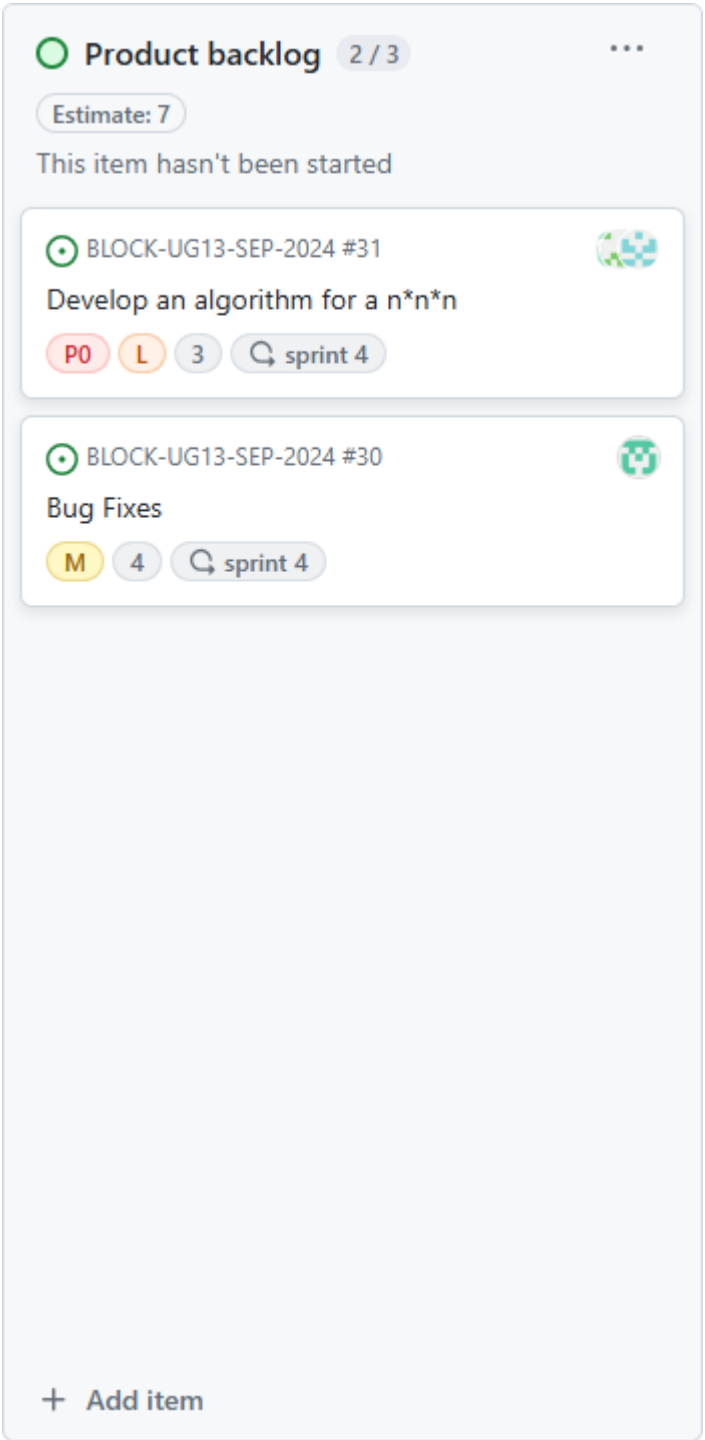
a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

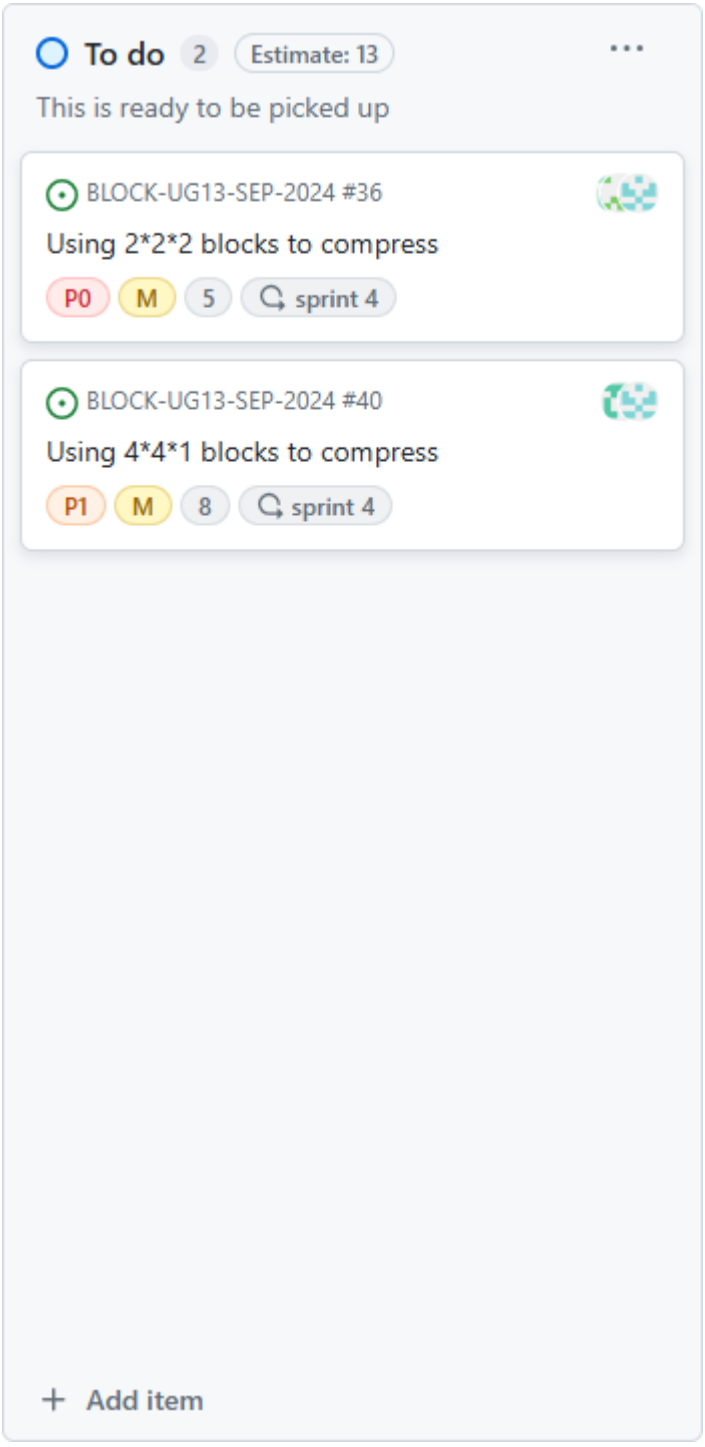
a1881772 Nauman Yawar Butt

Product Backlog and Task Board:

The product backlog



The task board



In progress 3 / 3 Estimate: 13

This is actively being worked on

BLOCK-UG13-SEP-2024 #18

Documentation Retrospective Sprint 4

P2 L 3 sprint 4

BLOCK-UG13-SEP-2024 #20

input stream processor update

P1 M 5 sprint 4

BLOCK-UG13-SEP-2024 #21

Error handling

P1 M 5 sprint 4

+ Add item

In review 3 / 5 Estimate: 12

This item is in review

BLOCK-UG13-SEP-2024 #33

Maptek competition

P0 L 5 sprint 4

BLOCK-UG13-SEP-2024 #39

Debug running shell, script

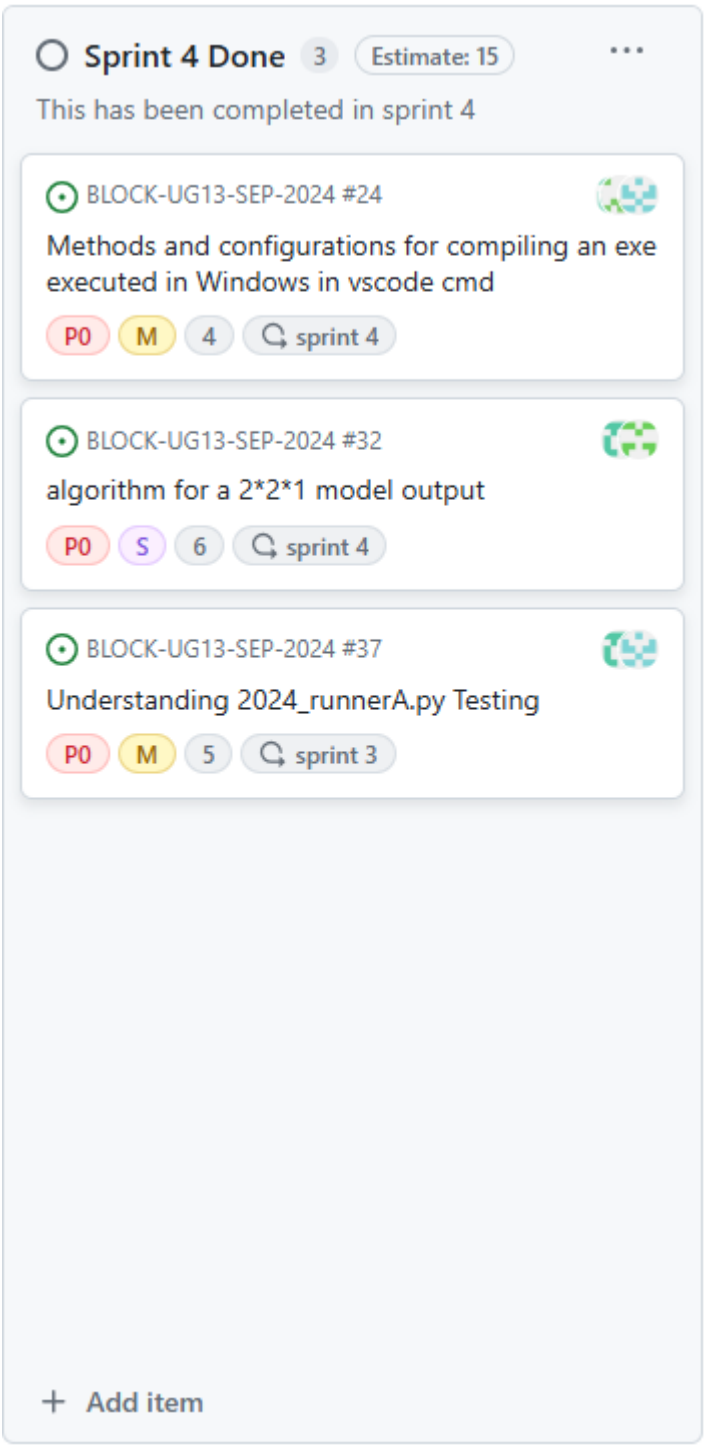
P0 M 5 sprint 4

Draft

Snapshot Week 9

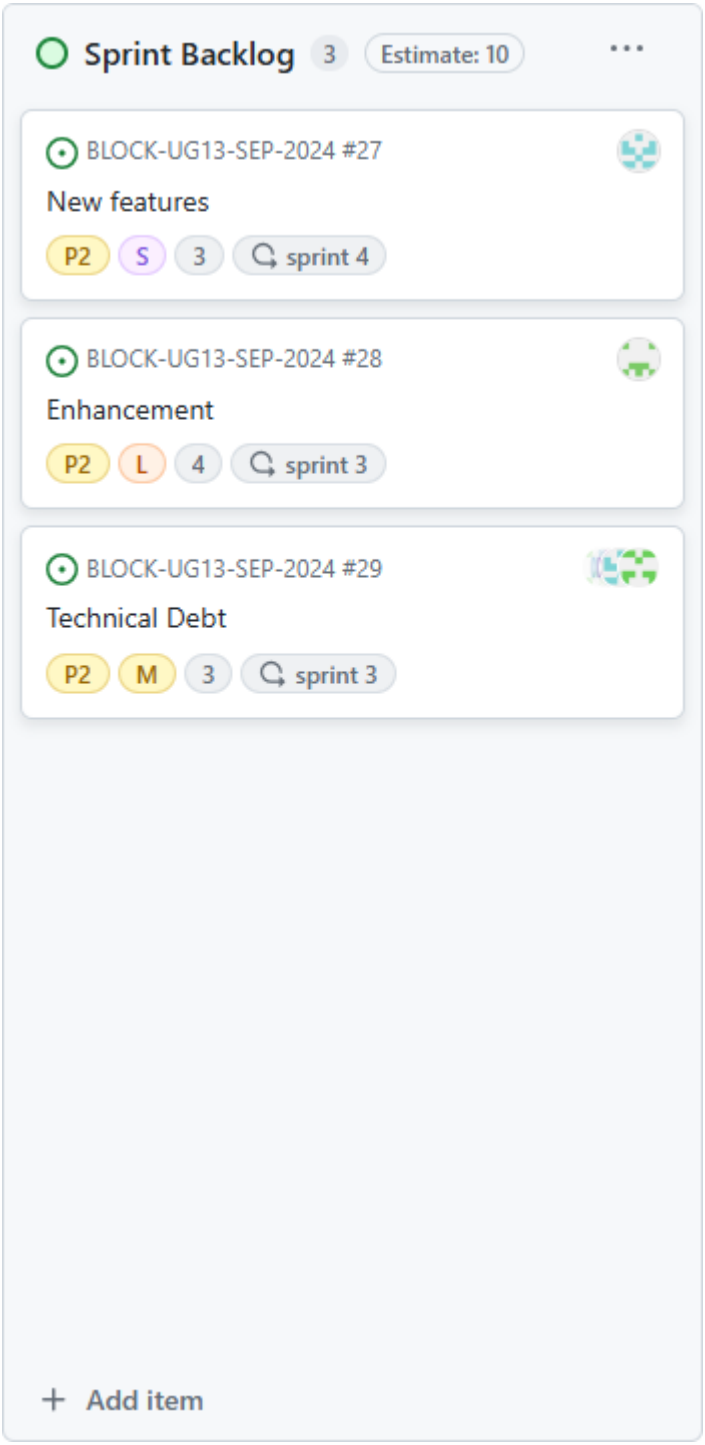
P2 L 2 sprint 4

+ Add item



Sprint Backlog and User Stories:

Sprint backlog



user stories

Sprint Overview

For the current sprint, we've chosen the following user stories to focus on:

1.As a product manager, we hope that the product can be launched as soon as possible, so we take the realization of the algorithm as the first goal in this sprint.

2.As developers, the testing process was very repetitive and tedious, so we wrote a debug script.

These user stories and their associated tasks will guide our current sprint development efforts, ensuring that we meet the needs of developers and customers as we create an effective block model compression program.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases

2 Tests passed:

The code passed the local test, conforming to logic and the output met

expectations.

Summary of Changes:

1. The code uses standard input (stdin) to read data instead of specifying a file name.

2. Modified the algorithm implementation method

Snapshot Week 10 of Group **Block-UG13**

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

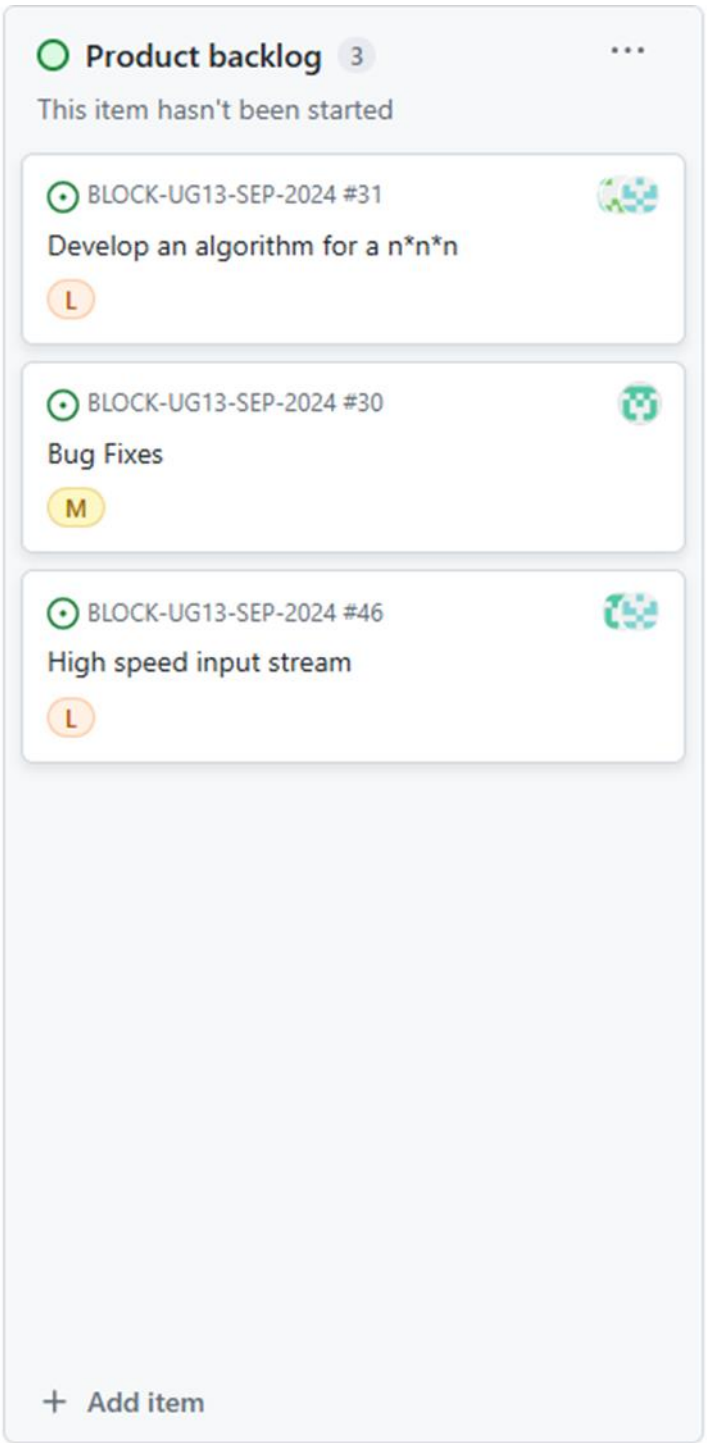
a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

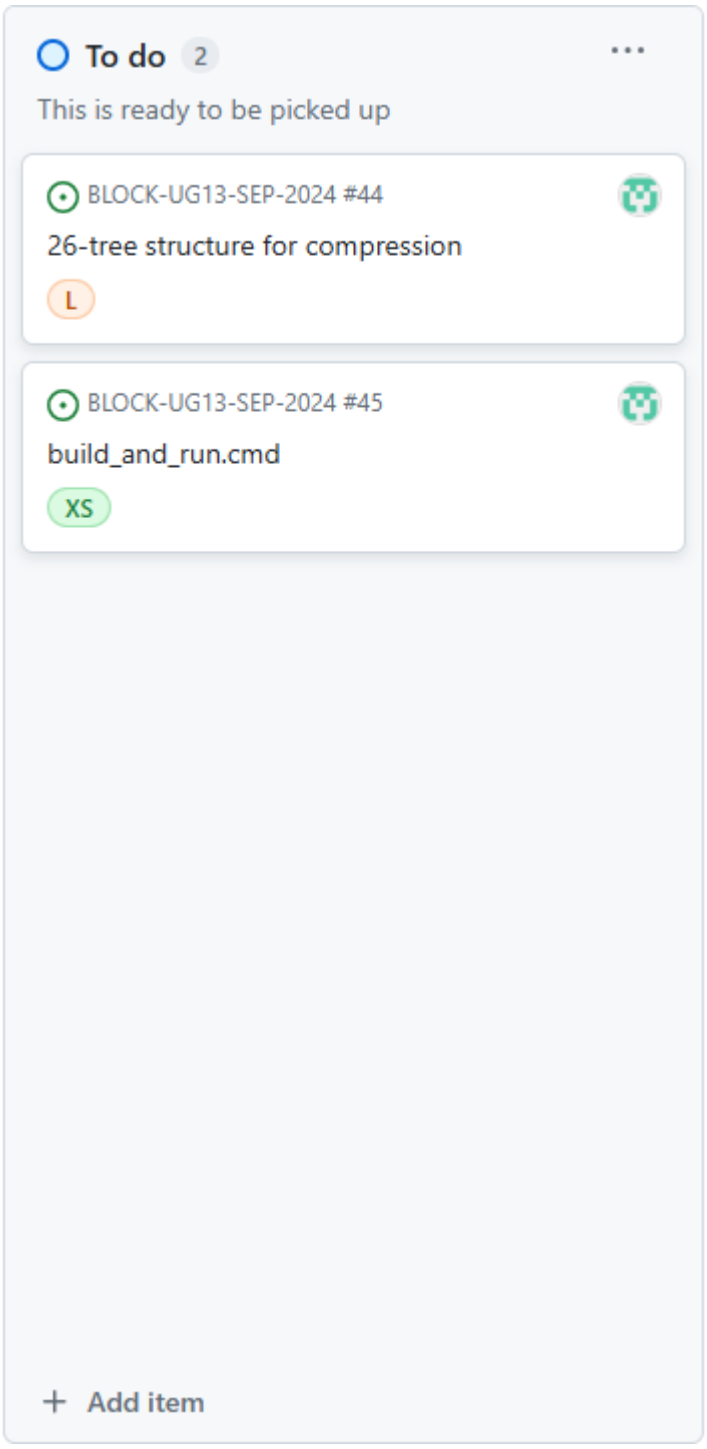
a1881772 Nauman Yawar Butt

Product Backlog and Task Board:

The product backlog



The task board



In progress 3

This is actively being worked on

BLOCK-UG13-SEP-2024 #21

Error handling

M

BLOCK-UG13-SEP-2024 #36

Using 2*2*2 blocks to compress

M

BLOCK-UG13-SEP-2024 #40

Using 4*4*1 blocks to compress

M

+ Add item

In review 2

This item is in review

BLOCK-UG13-SEP-2024 #18

Documentation Retrospective Sprint 4

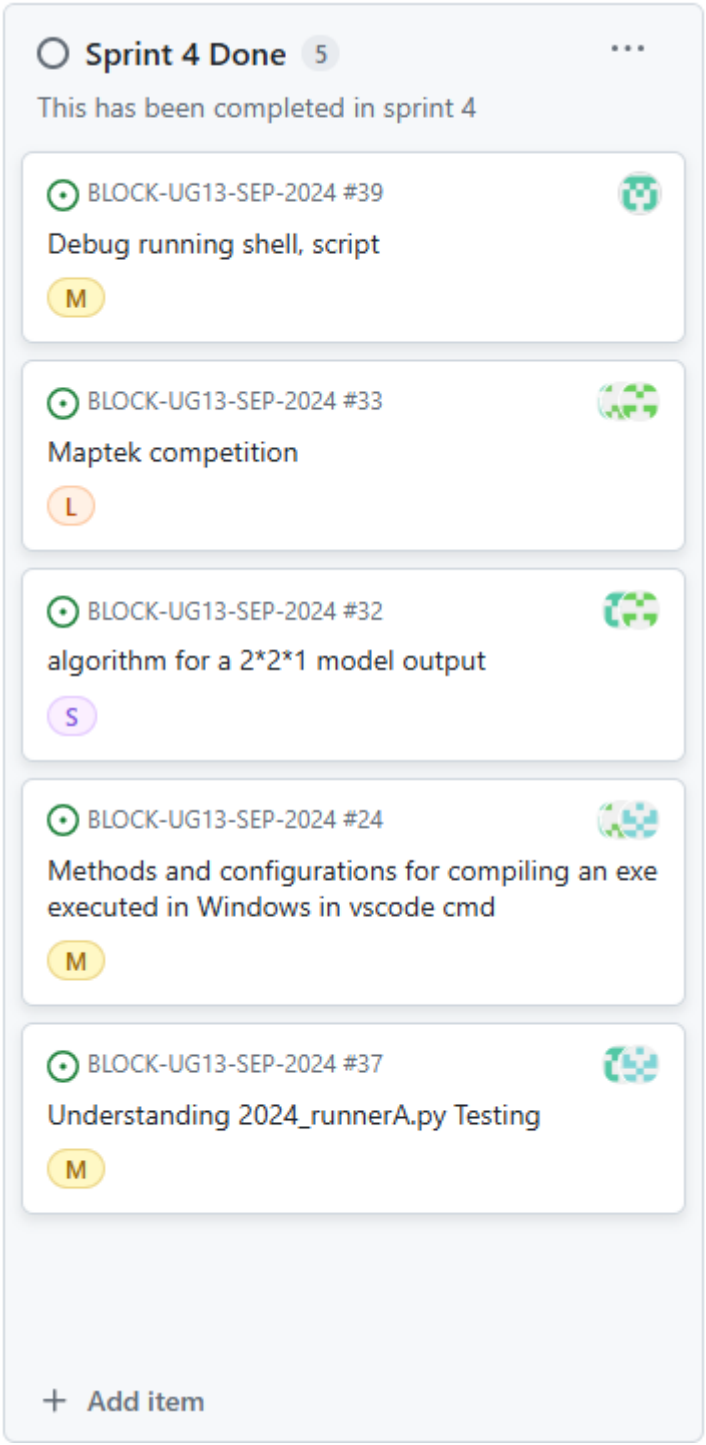
L

BLOCK-UG13-SEP-2024 #20

input stream processor update

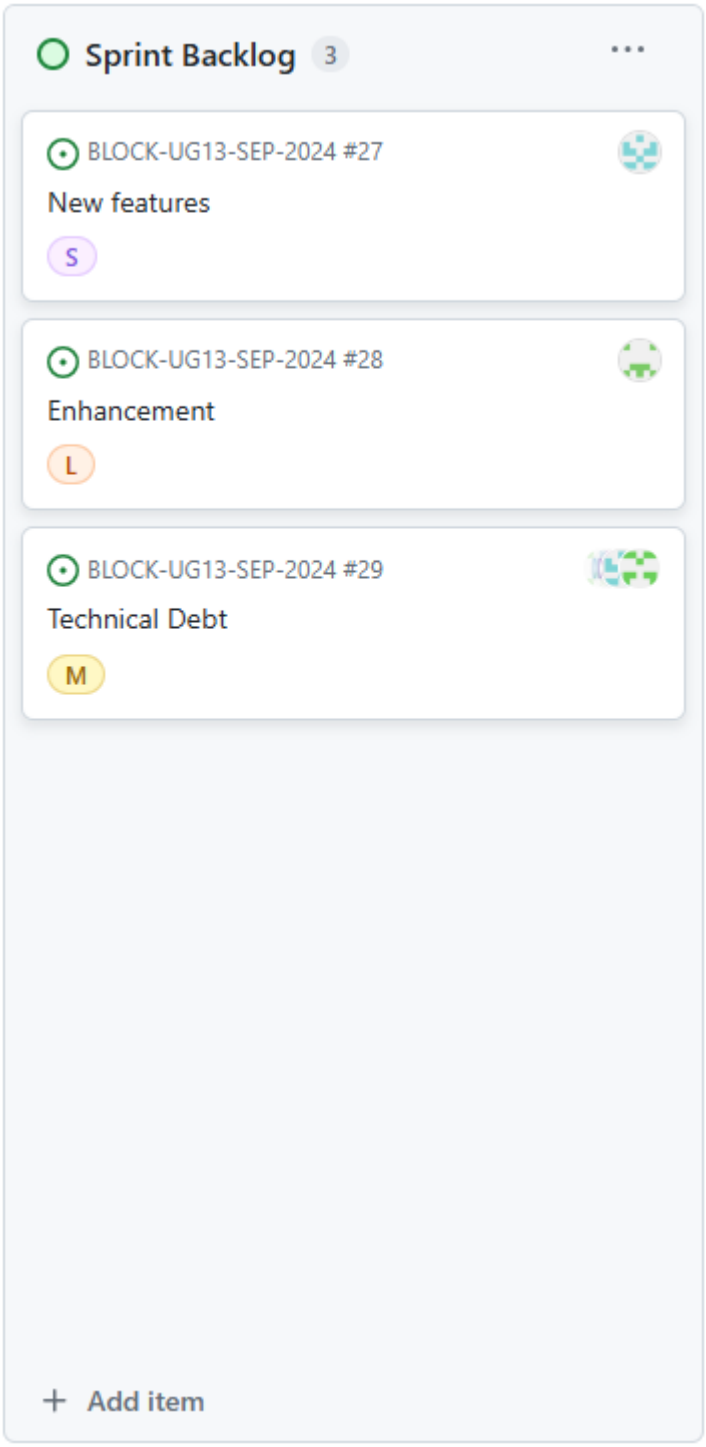
M

+ Add item



Sprint Backlog and User Stories:

Sprint backlog



user stories

Sprint Overview

For the current sprint, we've chosen the following user stories to focus on:

1. Since the last sprint, the focus of work of the team has been reassigned, and my work is mainly to write code. I wrote several versions and uploaded the website test, in which Block_UG13_1004.exe passed and was listed in the ranking。

2.As developers, the testing process was very repetitive and tedious, so we wrote a debug script:Build_and_run.cmd . After two weeks of testing, modification, and streamlining, it has been uploaded to Github for the group's partners to use.

These user stories and their associated tasks will guide our current sprint development efforts, ensuring that we meet the needs of developers and customers as we create an effective block model compression program.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases

2 Tests passed:

The code passed the local test, conforming to logic and the output met expectations.

Summary of Changes:

1. Modified the version, the performance is not up but down, no upload.

2. Debug running shell, script.

We first install VScode in Windows11, by opening virtualization and installing WSL and Ubuntu, install g++ compiler in Linux, exe compiled under Linux can not run in the website, but debugging and running more familiar and easy. So we first compile and run it under Linux to make the code bug-free. Because of the heavy debugging, we write the debugging process as a shell script. Perform batch data testing, compare compression rate and speed, and find performance improvement points.

3. To perform 3*3*3 compression, a data structure is required to easily obtain the state of 26 directional neighbor points.

Snapshot Week 12 of Group **Block-UG13**

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

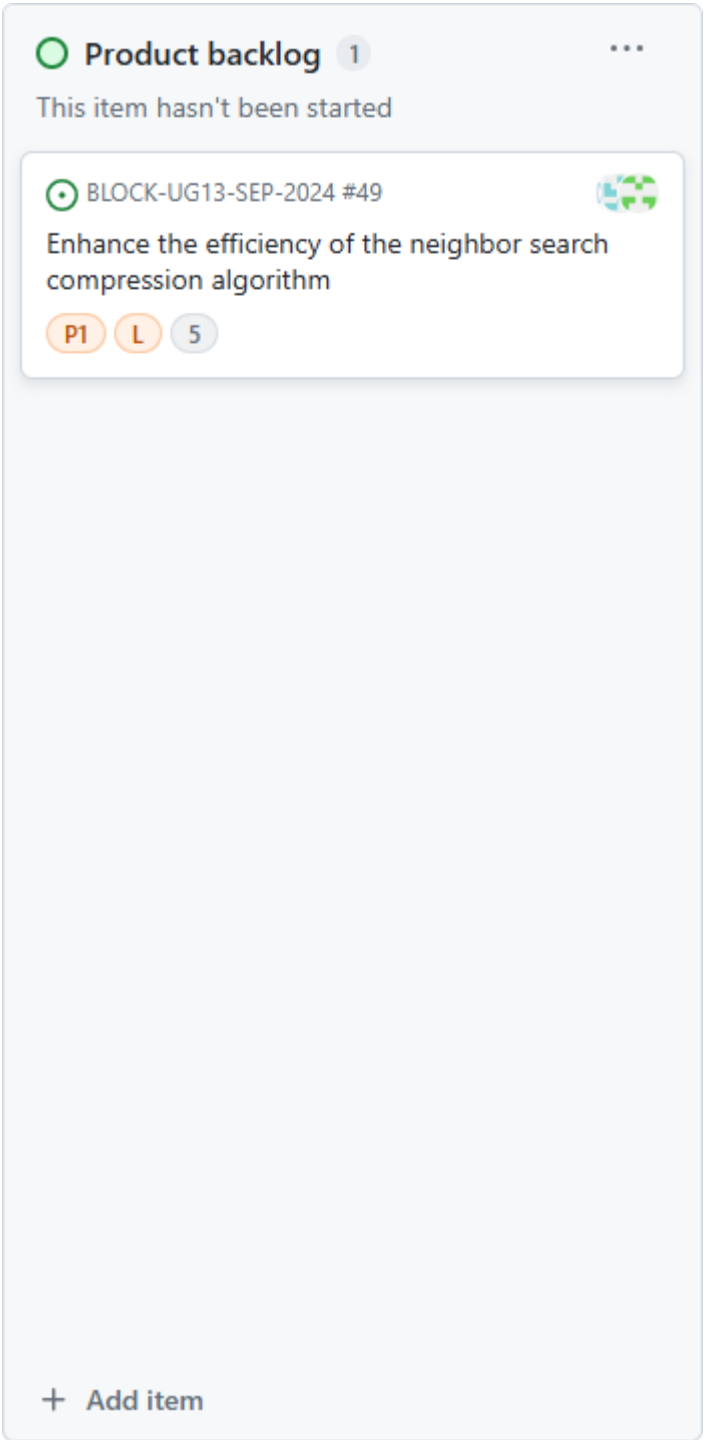
a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

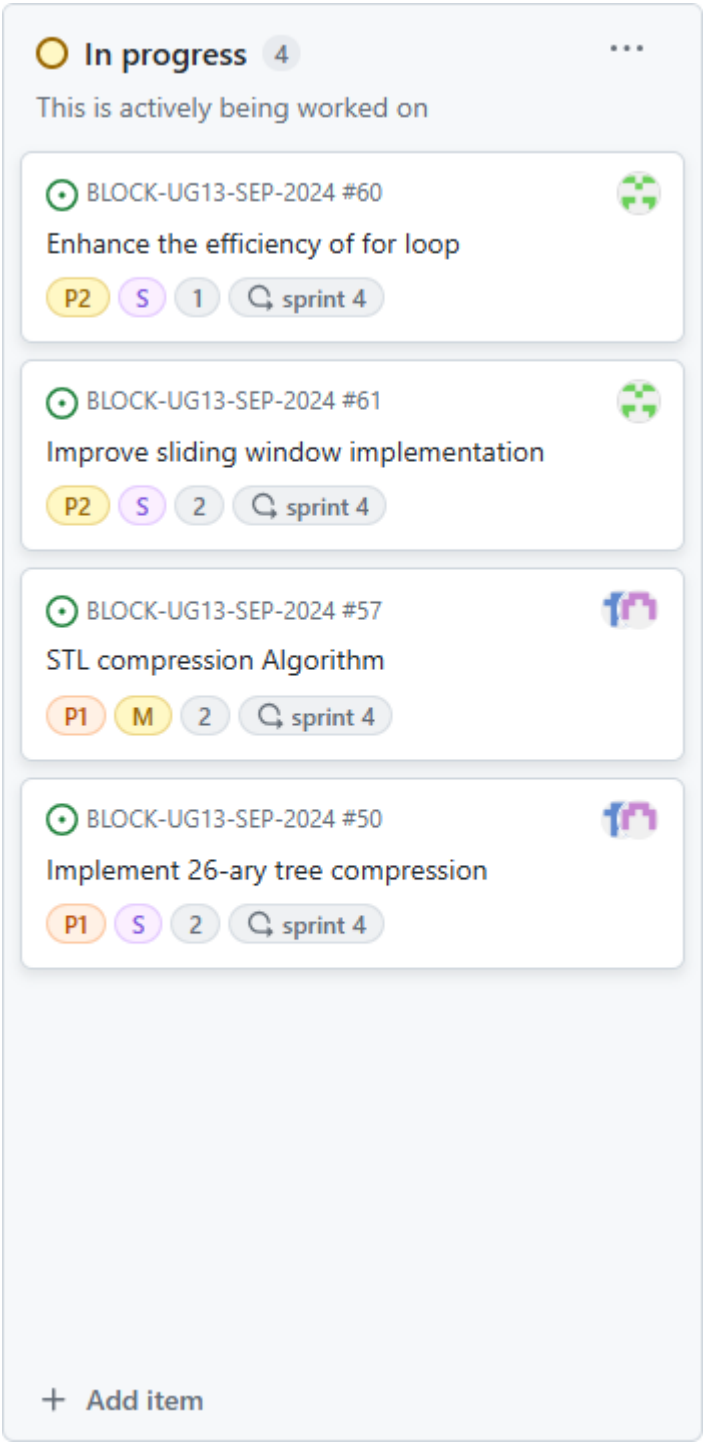
a1881772 Nauman Yawar Butt

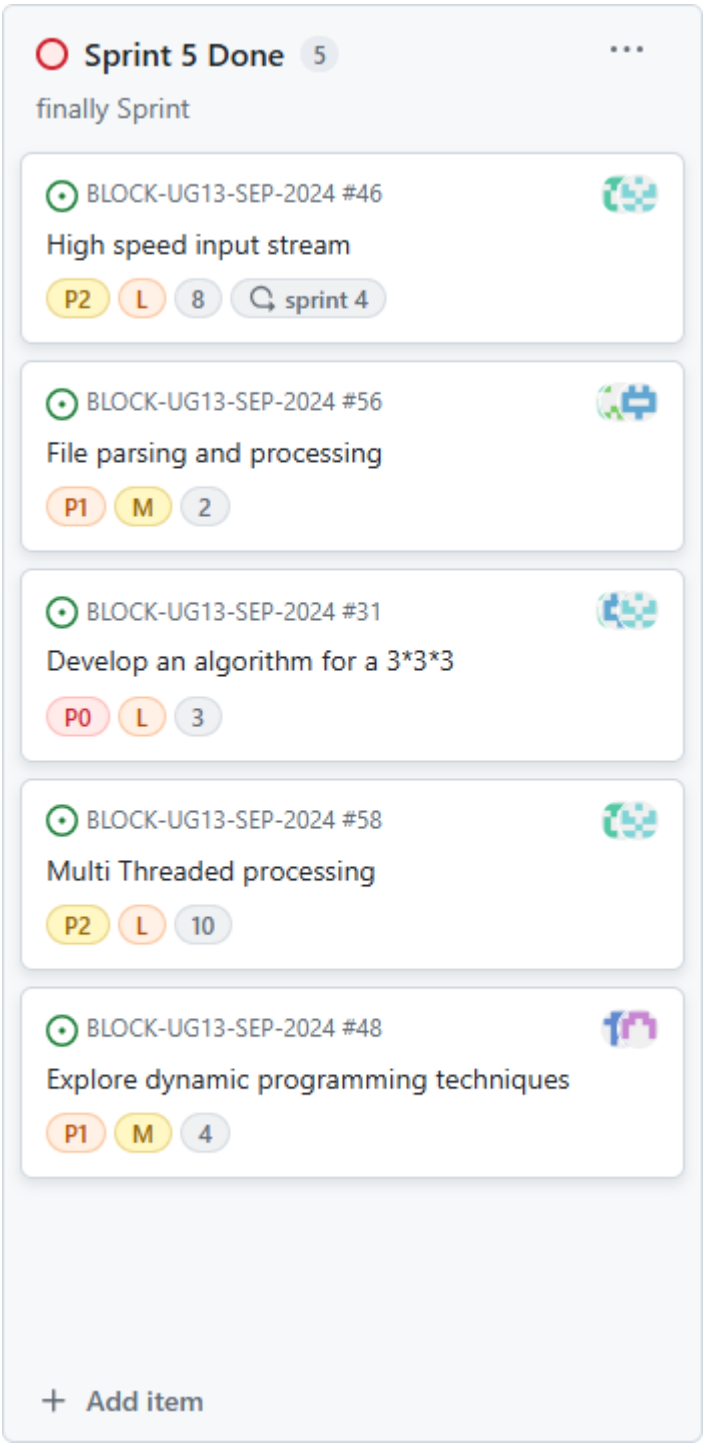
Product Backlog and Task Board:

The product backlog



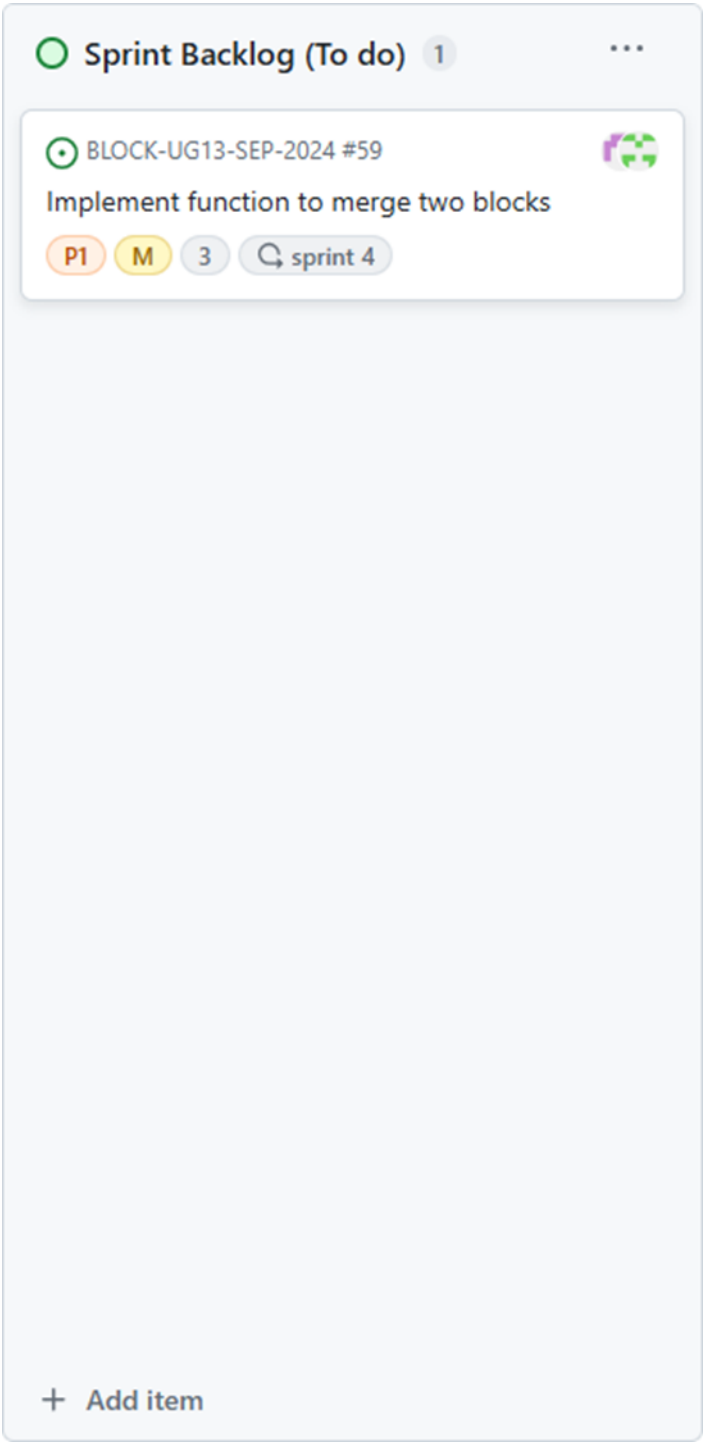
The task board





Sprint Backlog and User Stories:

Sprint backlog



user stories

Sprint Overview

For the current sprint, we've chosen the following user stories to focus on:

1. Firstly, we use Multi Threaded processing. Implementing the 2*2*1 algorithm with multithreading can significantly accelerate the compression process by parallelizing tasks, reducing computation time, and improving overall system efficiency.

2. Secondly, To test the compression of the Worldly One dataset, we achieved a ranking of 73.3849% on the Mapteh website, indicating that our algorithm performed well in terms of compression efficiency and speed. This result demonstrates the effectiveness of our technical stack and development standards.

3. Thirdly, To achieve efficient data compression, we can implement a 3D Run Length Encoding (RLE) algorithm, which encodes consecutive identical data points in 3D space, reducing redundancy and storage requirements.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases

2 Tests passed:

The code passed the local test, conforming to logic and the output met expectations.

Summary of Changes:

1 High speed input stream.

2 File parsing and processing.

3 Using utilizes a 26-way tree data structure , to implement a algorithm 3*3*3 blocks to compress.

4 Multi Threaded processing

5 Explore dynamic programming techniques

Snapshot Week 11 of Group **Block-UG13**

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

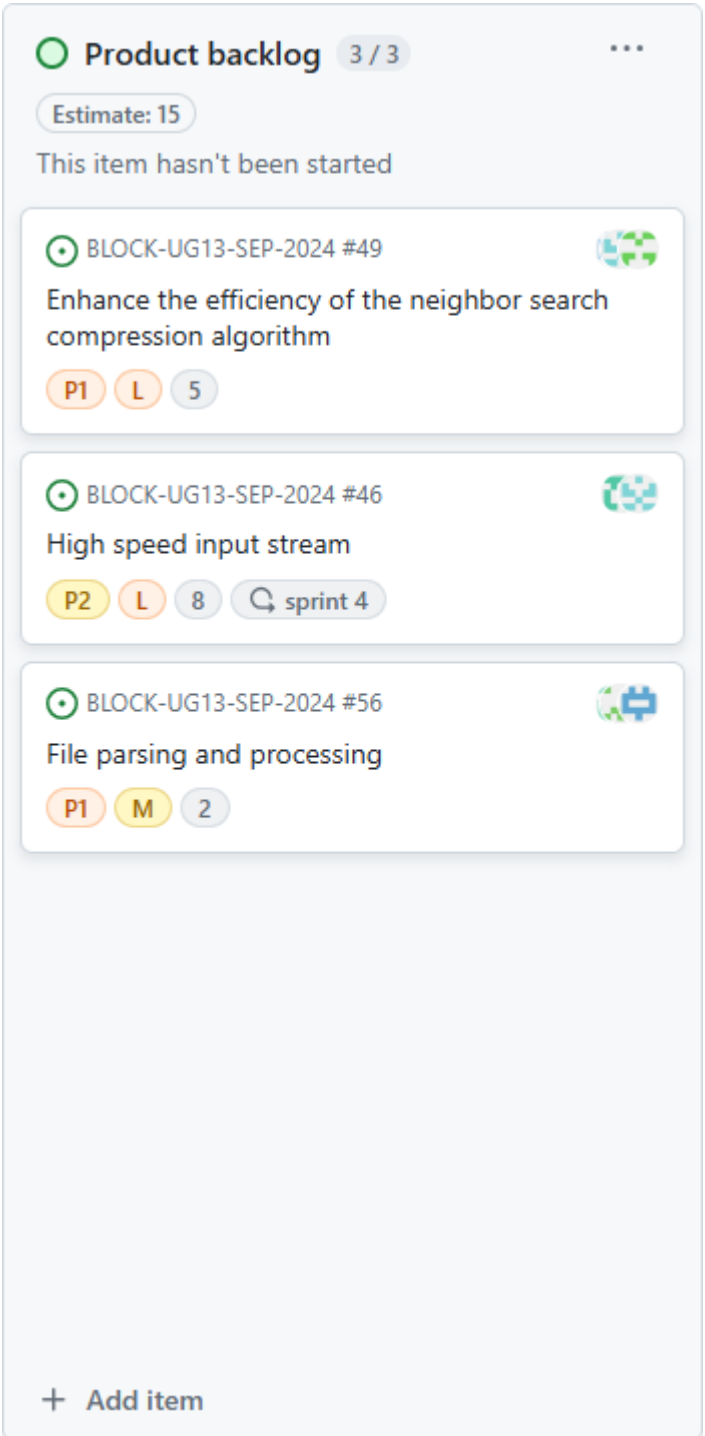
a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

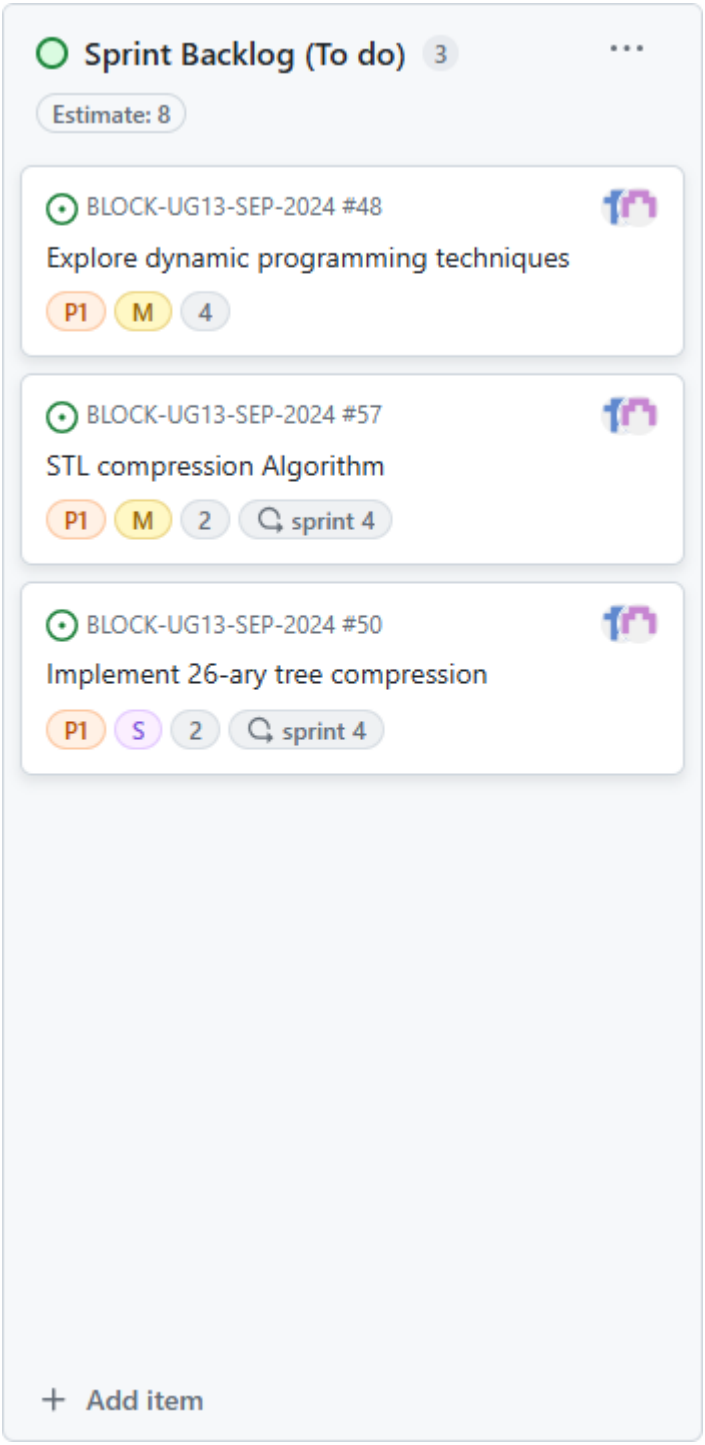
a1881772 Nauman Yawar Butt

Product Backlog and Task Board:

The product backlog



The task board



In progress 2 / 3 Estimate: 13

This is actively being worked on

BLOCK-UG13-SEP-2024 #31

Develop an algorithm for a 3*3*3

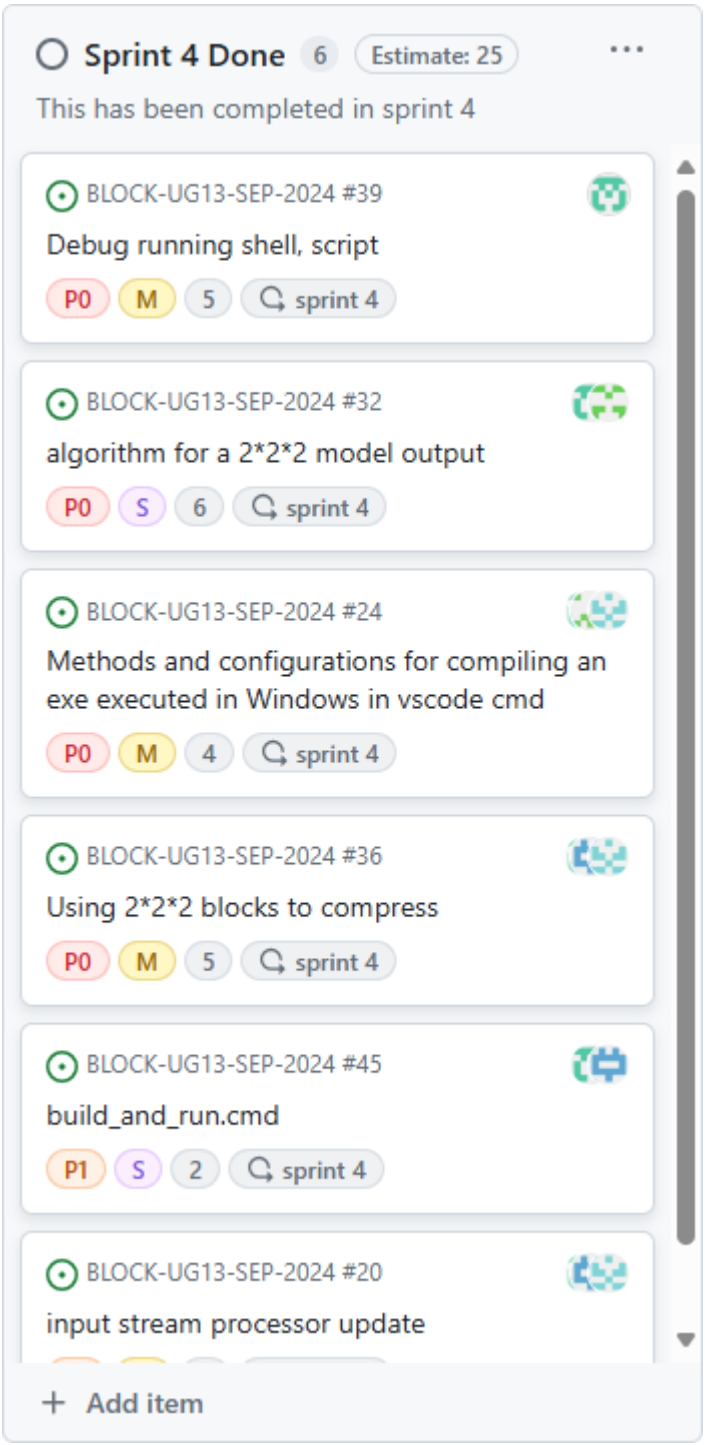
P0 L 3 sprint 4

BLOCK-UG13-SEP-2024 #58

Multi Threaded processing

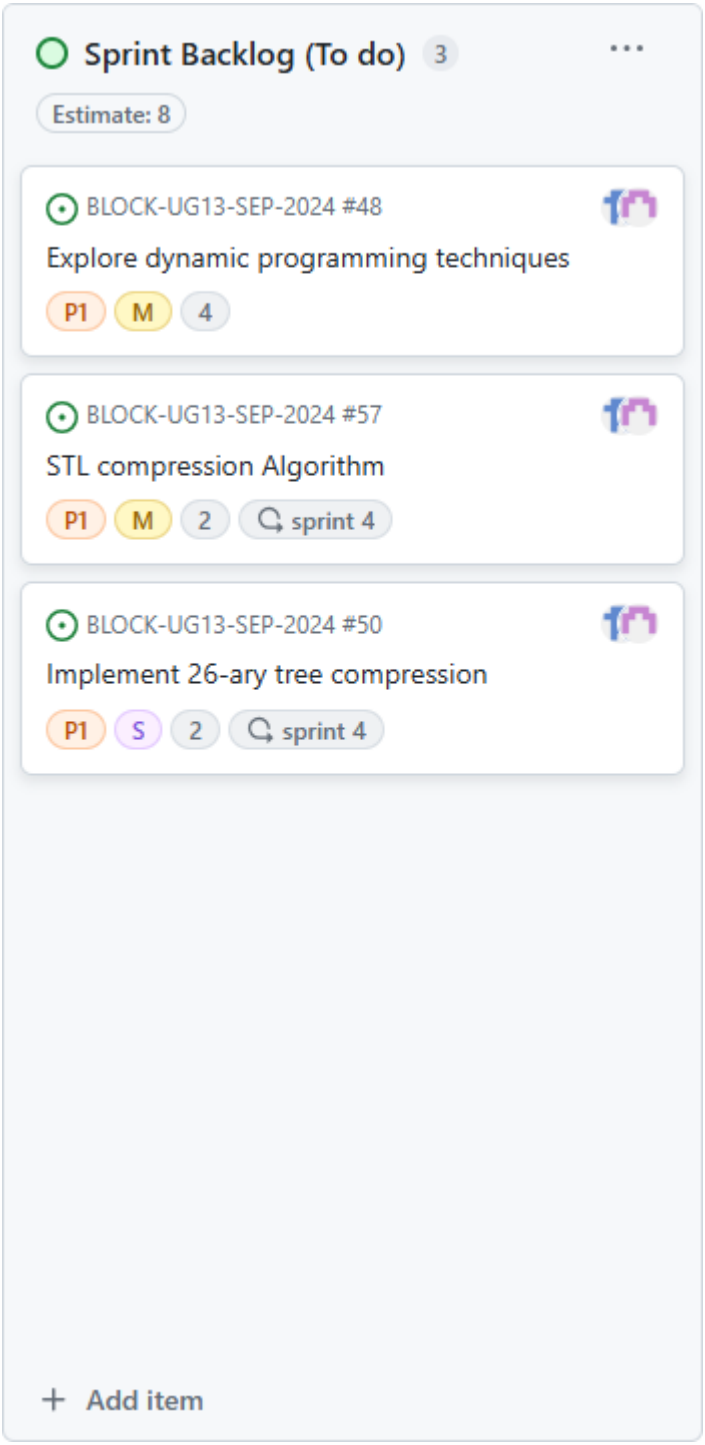
P2 L 10

+ Add item



Sprint Backlog and User Stories:

Sprint backlog



user stories

Sprint Overview

For the current sprint, we've chosen the following user stories to focus on:

1. Firstly, we resolved the issue where the code ran perfectly on local machines but encountered errors when deployed on the web server during the last sprint. We debugged the testing software, 2024_runnerA.py, and used this tool for code testing, greatly accelerating the debugging process.

2. Secondly, the 2*2*1 algorithm was uploaded to the mapteck web site for testing. The test was passed and a ranking was obtained. The test was passed in sequence for intro One, fast One, and real One. After improvement, Block_UG13_1013.exe passed the streaming One test, making some progress in both compression rate and speed.

3. Thirdly, we wrote the debugging process using shell scripts and had a clearer result in mind for the code uploaded to the website."

Moreover, we attempted to implement a 3*3*3 compression algorithm. This algorithm utilizes a 26-way tree data structure, which conveniently allows for the lookup of the 26 points connected to a certain point in the 3D structure, making the 3*3*3 compression operation very efficient.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases

2 Tests passed:

The code passed the local test, conforming to logic and the output met expectations.

Summary of Changes:

1 Debug running shell, script.

2 Write the algorithm of 2*2*2model compression uploaded to the mapteck web site.

3 Methods and configurations for compiling an exe executed in Windows in vscode/cmd

4 Using utilizes a 26-way tree data structure , to implement a algorithm 3*3*3

blocks to compress.

5 use build_and_run.cmd to debug and test.

6 input stream processor update.